

Dawid-Skene Server-Side Aggregation

Five-page logic and approval brief

11 August 2026

Branch: dawid-skene

1 Purpose

This document explains one contained implementation decision: use the Dawid-Skene algorithm on the central server to combine the hard labels that clients already upload. It is written so reviewers can understand the logic, the exact system boundary, and the controlled test before approving the work. **Status:** the logic is under review; it is not yet an implemented or validated result, and no performance improvement is claimed here.

Approval target. Approve a server-only implementation and a staged experiment. Majority voting remains available and remains the fallback. No client model, soft label, posterior, confusion matrix, or reliability score is transmitted.

2 Current decision and changed decision

Each client predicts a class for every sample in the same shared open dataset. A client sends a class number from 0 to 10, or -1 when it abstains. The server must turn these separate opinions into one hard global label per sample.

	Current: majority vote	With Dawid-Skene
Question asked	Which class received the most votes?	Which hidden class best explains the full pattern of agreements and disagreements?
Client treatment	Every accepted label counts equally.	Each client’s class-specific confusion pattern is estimated from the round’s label table.
Output	One hard label and one validity flag per sample.	The same hard label and validity flag.
Failure behavior	Deterministic majority result.	Deterministic majority result is used if the fit is unsupported, invalid, or does not converge.

Why it differs from majority

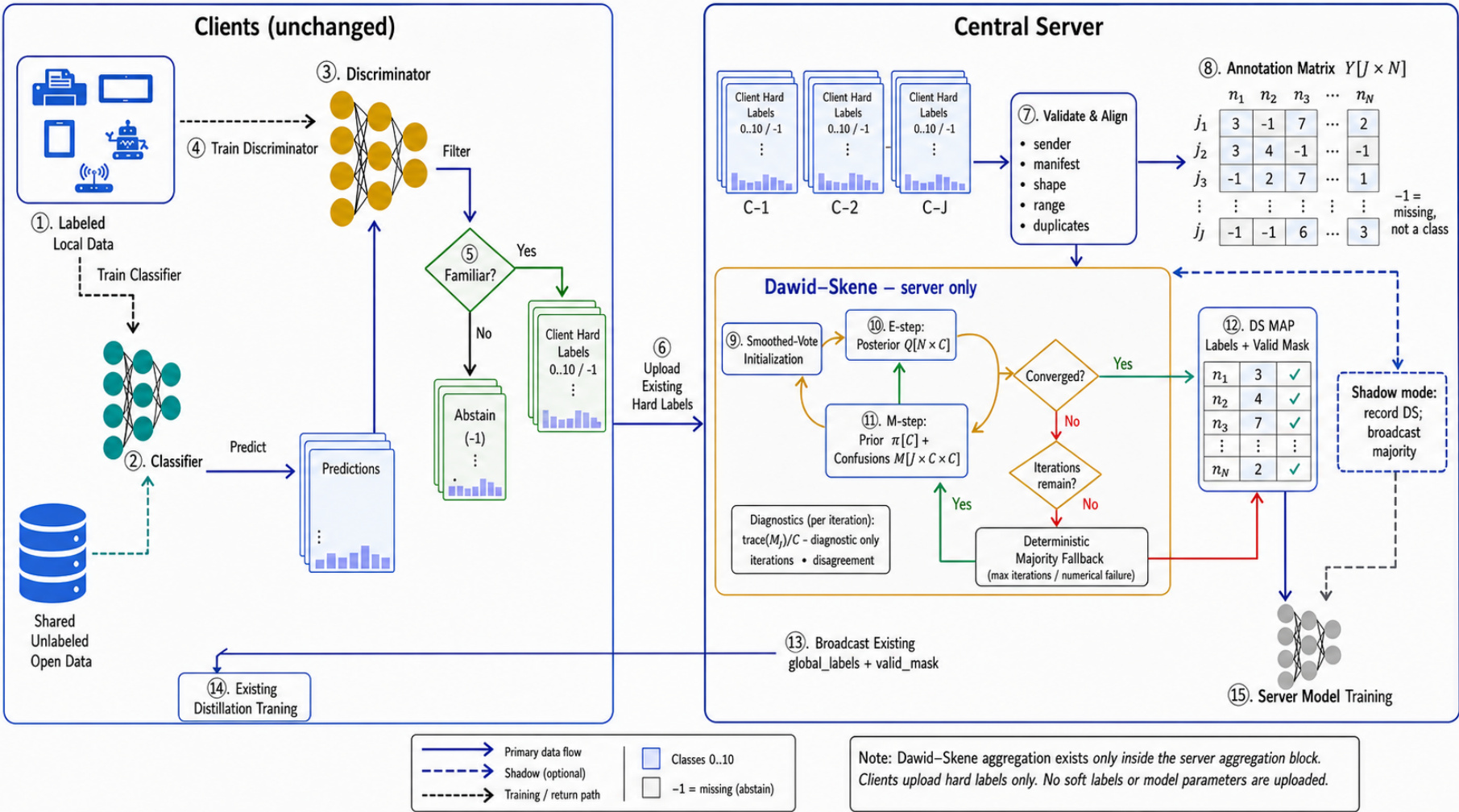
Clients do not necessarily make the same kind of mistake. One may distinguish classes 2 and 3 well but confuse classes 7 and 8; another may show the opposite pattern. Majority voting ignores that structure. Dawid-Skene estimates it and uses it when choosing the label. It can therefore choose a different class from majority when the disagreement pattern supports another class.

This does **not** make consensus equal to ground truth. Similar client models can make the same mistake together. The result must be checked offline against sealed labels after a run; sealed labels must never enter live aggregation or training.

Strict scope

- **Clients stay unchanged:** training, prediction, discriminator filtering, abstention, hard-label upload, and distillation are untouched.
- **The wire contract stays unchanged:** clients upload the existing hard-label vector; the server returns only `global_labels` and `valid_mask`.
- **Only the server decision changes:** Dawid-Skene is inserted where the server currently combines validated hard-label proposals.

System flow and exact Dawid-Skene boundary



How to read the figure. Steps 1–6 on the client side are the existing workflow. The only new calculation is the orange server block: build the client-by-sample label matrix, alternate the E-step and M-step, and choose hard maximum-posterior labels. A -1 entry means “missing”, not a class. Shadow mode records the Dawid-Skene answer but broadcasts majority. Active mode broadcasts the Dawid-Skene hard labels only after the fit succeeds. A numerical failure or an unconverged iteration cap returns to majority. Downstream client distillation and server-model training are outside the Dawid-Skene loop.

3 Algorithm logic

Let Y_{ji} be client j 's label for shared sample i . Legal observations are classes $0, \dots, C-1$; $Y_{ji} = -1$ is missing. The server estimates:

- $P(z_i = c)$: the posterior probability that sample i belongs to hidden class c ;
 - π_c : the estimated frequency of class c ; and
 - $M_j[c, k]$: how often client j reports class k when the hidden class is c .
1. **Validate and align.** Check sender identity, manifest, vector shape, legal values, and duplicates. Sort accepted senders so input order cannot change the answer. Identical retries count once; conflicting replies from one sender are rejected for that round.
 2. **Build the matrix.** Put accepted vectors into $Y[\text{client}, \text{sample}]$ and treat every -1 as missing evidence.
 3. **Initialize.** Start sample probabilities from smoothed vote proportions. Smoothing prevents an unseen class from receiving an irreversible probability of zero.
 4. **M-step: reconsider clients.** Use the current sample probabilities to update the class prior and every supported row of every client's confusion matrix. Pseudocounts prevent zero rows and stabilize sparse classes.
 5. **E-step: reconsider samples.** For every usable sample and class, combine the updated class prior with the observed clients' current confusion probabilities:

$$P(z_i = c \mid Y_{\cdot i}) \propto \pi_c \prod_{j: Y_{ji} \neq -1} M_j[c, Y_{ji}].$$

The implementation performs this calculation in log space to avoid numerical underflow.

6. **Repeat.** Alternate M and E until the regularized objective changes by no more than the tolerance, or the iteration ceiling is reached.
7. **Return a hard decision.** For every supported sample, choose the class with the largest posterior probability. Exact ties use the lowest class index, matching deterministic behavior.

Operational pseudocode

```
collect and validate existing client hard-label vectors
compute deterministic majority once for fallback
if there is insufficient usable evidence: return majority
initialize posterior probabilities from smoothed votes
repeat M-step then E-step in float64 log space
    if any value is invalid or the objective falls beyond numerical tolerance: return majority
    if converged: return hard MAP labels and the existing valid mask
if the iteration ceiling is reached without convergence: return majority
record mode, support, iterations, likelihood, objective, timing, disagreement, and fallback reason
```

Important interpretation. The estimated confusion matrices are round-specific aggregation variables, not permanent client reputation scores. The first implementation starts cold in every round; warm start is disabled.

4 Implementation and safety contract

The insertion point is `SSFLStrategy.aggregate_train` in `src/ssfl/strategies/ssfl.py`. After the server validates and de-duplicates client proposals, the current hard-label branch calls majority aggregation. The new server selector chooses majority, shadow, or active Dawid-Skene and always returns the existing aggregation result shape. The existing soft-probability research path is separate and is not changed by this work.

Server area	Required change
Estimator	Add a self-contained hard-label Dawid-Skene calculation: initialization, E-step, M-step, stopping checks, a success/failure result, and aggregate diagnostics.
Aggregation selector	Build the stable client-by-sample matrix; preserve the current majority function and lowest-class tie-break; select majority, shadow, or active mode. When no valid proposal exists, preserve the current outer control flow and do not invoke Dawid-Skene.
Configuration	Add validated settings, include them in the run identity, and reject unsupported warm start. Majority remains the default.
Diagnostics and reports	Record convergence, fallback, disagreement, support, time, and memory; keep results separated by aggregation mode. Client confusion matrices remain restricted diagnostics.

Settings to lock before active testing

Setting	Starting value	Meaning
Mode	shadow	Compute Dawid-Skene but broadcast majority first.
Maximum/minimum iterations	100 / 2	Stop a stalled fit; prevent stopping at initialization.
Relative tolerance	10^{-6}	Required regularized-objective change for convergence.
Initialization	smoothed votes	Encourage alignment with submitted class indices; this does not guarantee against label switching.
Initialization/confusion/class-prior pseudocounts	0.01 / 0.1 / 1.0	Stabilize initial, client-confusion, and class-prior probabilities.
Minimum clients	3	Practical support guard before fitting; not a proof of identifiability.
Minimum item/client annotations	1 / 1	Keep primary coverage equal to the present hard-label path.
Posterior threshold/damping	0.0 / 1.0	Do not win by rejecting samples; use the full update.
Precision/epsilon/warm start	float64 / 10^{-12} / false	Stable arithmetic; independent fit in every round.
Failure policy	majority	Includes invalid arithmetic and cap without convergence.

These are initial engineering values for review, not defaults reported by Dong et al.

Non-negotiable safeguards

- No new Dawid-Skene field may cross the network. Shadow mode must broadcast the same logical labels and mask as majority for the same valid unique-sender submissions.
- Insufficient support, non-finite values, objective failure, or reaching the cap without convergence must use majority from the same validated and de-duplicated proposal batch.
- Raw client-by-sample matrices are off by default; any temporary audit is restricted, pseudonymized, owner-controlled, and deleted after settings are locked.

5 Controlled check and approval decision

Test sequence

Stage	Execution	Purpose
Unit and bench- mark gates	Local fixtures at 27- and 89-client sizes	Verify hand-computed E/M results, missing labels, sender- order invariance, numerical failure, fallback, memory, and latency.
Shadow	All three scenarios; ma- jority remains broadcast	Give majority and Dawid-Skene exactly the same live propos- als. Check convergence, disagreement, latency, memory, and the unchanged payload before any Dawid-Skene label affects training.
Matched active comparison	Majority versus Dawid- Skene in all three sce- narios and multiple inde- pendent seeds	Lock settings first. Pair dataset manifest, split, scenario, seed, initial checkpoint, client participation policy, model, training settings, round count, code, and hardware. The only assigned intervention is hard-label aggregation; later model and pro- posal differences are downstream treatment effects.

What will be measured

- **Primary model outcomes:** mean server macro-F1 and accuracy over the final 10 rounds.
- **Offline label quality:** accuracy, macro-F1, per-class recall, corrected errors, introduced errors, matched coverage, posterior calibration, and class balance.
- **Safety:** worst-class recall, new class collapse, fallback rate, convergence, invalid values, and whether gains come mainly from rejecting samples.
- **Engineering:** server aggregation time, peak memory, audit size, resume behavior, and proof that tensor names, shapes, types, and logical network byte counts are unchanged.

Results are paired within scenario and seed, and scenarios are reported separately. Items and rounds are repeated observations, not independent experiments. Every assigned active round remains in the active arm even if it falls back to majority. Settings are locked before the matched comparison and are never tuned using sealed test labels.

Stop conditions

Stop and inspect before continuing if probabilities are invalid, hidden class meanings switch, a class collapses, convergence repeatedly fails, fallback becomes frequent, runtime or memory grows materially, any new client payload appears, or matched results are worse. All outcomes are reported; failed runs are not replaced with better seeds.

Reviewer approval requested: mark Yes, No, or Changes required.

- | | |
|---|--|
| 1. Server hard-label aggregation seam only; no client or message-schema change. | ☐ Yes ☐ No |
| 2. EM logic and initial settings on pages 3–4, including -1 as missing. | ☐ Yes ☐ No |
| 3. Cold start each round and majority fallback for unsupported or failed fits. | ☐ Yes ☐ No |
| 4. Shadow execution before any Dawid-Skene label is broadcast. | ☐ Yes ☐ No |
| 5. Matched majority-versus-Dawid-Skene controls and reported metrics above. | ☐ Yes ☐ No |

Required changes: _____

Approved by: _____

Date: _____

Reference and scope note

J. Dong, R. Zhu, X. Shang, and J.-H. Xue, “Dawid-Skene-model-based label-noise mitigation for federated learning,” *Information Sciences*, vol. 745, art. 123425, 2026, [doi:10.1016/j.ins.2026.123425](https://doi.org/10.1016/j.ins.2026.123425). The paper’s FedDS workflow uploads client model parameters and weights model aggregation. This brief does not copy that protocol. It uses Dawid-Skene only to combine the hard labels already available on the server, so the current client and network boundary remains intact.